

南開大學
Nankai University

南　　开　　大　　学

计　算　机　学　院

编译原理上机实验报告

预备工作：了解你的编译器、LLVM IR 与汇编编程

姓名：张龙飞、袁煜杰

学号：2413625、2412303

专业：计算机科学与技术

2026 年 10 月 11 日

摘要

本实验通过编译工具的阶段输出研究语言处理流程，并设计六组 SysY 示例，编写等价的 LLVM IR 与 RISC-V 汇编，涵盖算术、分支、循环、函数、数组和浮点转换。通过参考程序、本机及 RV64 运行对照，共 269 项检查全部通过。进阶部分结合 AscendNPU IR 官方 VecAdd 文档分析方言层级，并执行上游 MLIR 从 Linalg 经 SCF、CF、LLVM 方言到 LLVM IR 的降低，CPU VecAdd 的 16 个输出元素均正确。

关键词：SysY；LLVM IR；RISC-V；SSA；MLIR；等价性验证

目录

一、 引言 **1**
（一） 实验背景与目的 1
（二） 小组分工 1
（三） 工作流程实验环境 1
（四） IR 与 MLIR 实验环境 2

二、 基础要求 **2**
（一） 了解编译器工作过程 2
（二） 熟悉 LLVM IR 中间语言 7
（三） 熟悉汇编语言 11

三、 进阶要求 **15**
（一） 观察 AI 编译器 MLIR 中端处理过程 15

四、 总结与体会 **19**

一、 引言

（一） 实验背景与目的

SysY 是全国大学生编译系统设计赛定义的简化 C 语言。本课程上机大作业要求实现一个 SysY 编译器（词法分析、语法分析、类型检查与中间代码生成、代码优化、目标代码生成），目标平台为 RISC-V。本次预备工作是上机大作业的第一个模块，目的在于：

1. 以 GCC / LLVM(Clang) 为研究对象，深入探究语言处理系统的完整工作过程，理解预处理器、编译器（前端与后端各阶段）、汇编器、链接器的功能；
2. 熟悉 LLVM IR 中间语言与 RISC-V 汇编语言，设计覆盖 SysY 语言特性的示例程序，手写等价的 LLVM IR 与 RISC-V 汇编程序，链接 SysY 运行时库并验证运行结果的正确性；
3. 为后续词法分析器、语法分析器、中间代码生成与目标代码生成等模块的实现建立直观认识与技术储备。

（二） 小组分工

本实验由两人合作完成，成员与分工如表 1 所示。基本要求一（编译系统工作流程研究）与进阶要求（MLIR 多级中间表示探索）由两人共同完成；基本要求二按分工分别编写 LLVM IR 与 RISC-V 汇编，实验结果合并展示。报告的框架、摘要、引言与结论由两人共同撰写。

表 1 小组成员与分工

成员	学号	承担工作
张龙飞	2413625	基本要求一（共同完成）；基本要求二：RISC-V 汇编编程
袁煜杰	2412303	基本要求一（共同完成）；基本要求二：LLVM IR 编程

（三） 工作流程实验环境

本实验在 WSL2（Windows Subsystem for Linux）中进行，使用 QEMU 用户态模拟器运行 RISC-V 目标程序。主要工具及版本如表 2 所示。

表 2 实验环境与工具版本

项目	版本 / 说明
操作系统	WSL2：Ubuntu 22.04.5 LTS（x86_64）
GCC / G++	11.4.0
Clang / LLVM 工具链	14.0.0（含 llvm-as、lli、llc、opt、llvm-dis）
Flex / Bison	2.6.4 / 3.8.2
RISC-V 交叉工具链	riscv64-linux-gnu-gcc / g++ 11.4.0
模拟器	QEMU 7.0.0（qemu-riscv64 用户态模式，/opt/qemu 源码编译）

（四） IR 与 MLIR 实验环境

实际运行环境为 WSL Ubuntu 26.04.1 LTS（x86_64），Clang/LLVM 21.1.8，RISC-V GCC 15.2.0，QEMU 10.2.1。MLIR 21.1.8 工具从 Ubuntu 官方源下载并在项目内解压。详细命令和版本保存在实验结果目录中。

二、 基础要求

（一） 了解编译器工作过程

1. 实验程序与步骤

选用阶乘程序作为实验对象，输入 n，输出整数阶乘。验证输入限制在 $0 \le n \le 12$，覆盖零次循环、普通循环与较深循环。

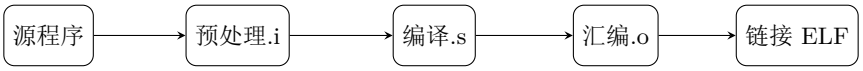


图 1 从源程序到可执行文件的主要阶段

表 3 阶段观察方法

阶段	实验命令选项	主要产物
预处理	`clang++ -E`	.i
词法分析	`-Xclang -dump-tokens`	token 文本
语法分析	`-Xclang -ast-dump`	AST 文本
语义检查	`clang++ -fsyntax-only`	错误诊断
中间代码	`clang++ -S -emit-llvm`	.ll
优化	`opt -passes=mem2reg,instcombine`	优化后的.ll
代码生成	`g++ -S`；交叉工具链 `-S`	x86 / RV64 .s
汇编	`clang++ -c fact_explicit.s`	可重定位.o
链接	`g++`；对照 `-static`	可执行 ELF
观察工具	`objdump -d/-r/-S`、`nm`、`readelf`、`ldd`	符号、重定位等

2. 预处理：头文件、宏与条件编译

`fact.cpp` 共 17 行，预处理输出 `fact.i` 共 30194 行。新增内容主要来自 `iostream` 及其递归包含的类型、函数和模板声明；文件开头的行标记用于关联展开内容的原始文件位置。预处理不计算阶乘，而是在词法及语法处理前展开包含文件、宏和条件编译分支。为单独观察宏与条件编译，另写一个不含头文件的宏实验：将 STEP(x) 定义为 ((x)+1)，由 MODE 决定保留 STEP(1) 还是 STEP(4)。

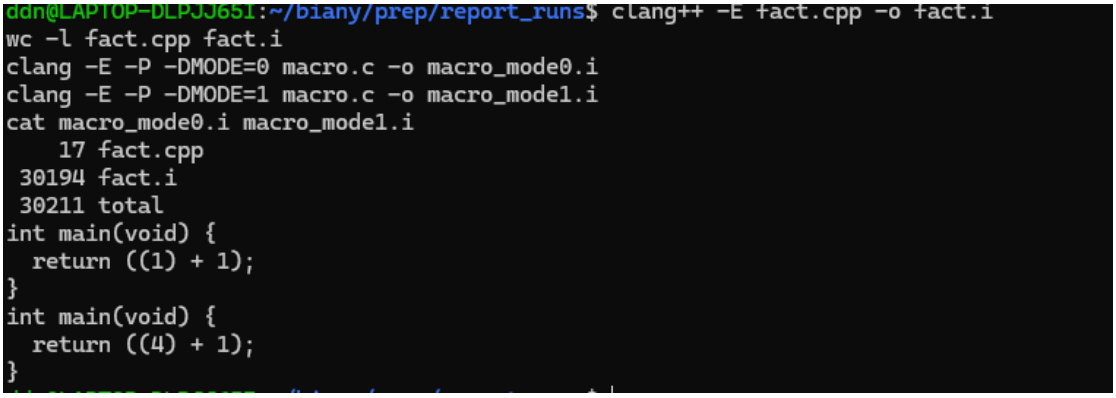


图 2 预处理与条件编译结果

两份实际输出均不含原注释、`#define` 或 `#if` 指令；`STEP` 被替换为带括号的表达式，`-DMODE` 决定保留哪个源分支。`-P` 仅移除行标记以方便比较，并不改变程序计算。

3. 编译器前端

词法分析 token 转储共 126745 行，包含头文件 token 和 EOF。每行记录类别、拼写及源位置。例如 cin、»、n、分号分别对应 identifier、greatergreater、identifier、semi。

词法阶段识别名字、数字、关键字与操作符，但不会根据名字是否声明判断程序正确性。`>>` 被作为一个多字符 token；它在此处最终调用流提取运算符，该含义需要后续类型与重载解析确定，不能仅根据 token 拼写认为它执行整数右移。

语法分析 使用 `-ast-dump-filter=main` 过滤标准库树节点，观察 main 的结构。

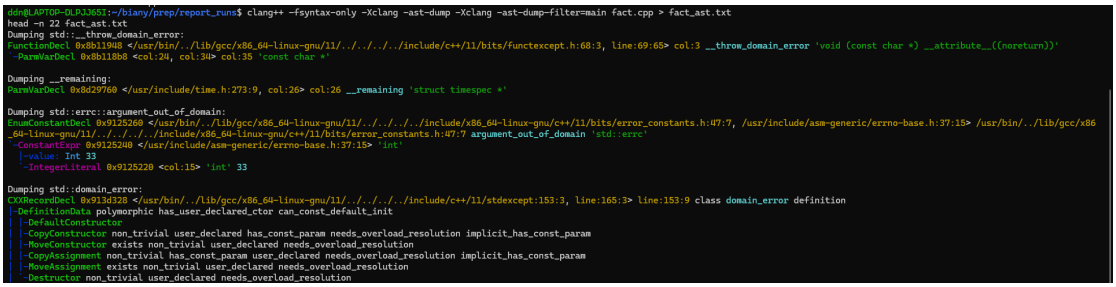


图 3 main 的 AST 结构

源码中的语句顺序对应语句块的子节点顺序，while 包含条件及循环体，乘法位于赋值右侧。树结构表达了嵌套关系与优先级。Clang 的 AST 转储同时携带类型、隐式转换和引用信息，因而并非只有语法分析结果，也已经体现部分语义分析。

语义检查与语法错误的区别 编写四个独立小程序，使用 `-fsyntax-only` 运行前端检查。该参数包含语法与语义检查，不能将全部诊断都归为语法错误。

参数类型错误用例 `square("abc")` 具有合法的函数调用结构，但字符串不能转换为形参要求的 int。编译器给出候选函数不可用、`const char[4]` 无法转换为 int 的诊断。未声明变量的程序同样可以形成语法结构，却无法完成名字绑定；`int x = ;` 则在构造完整表达式时就失败。三个反例说明类型和名字检查需要符号与上下文信息，而不能仅依赖 token 序列。

表 4 前端错误分类实验

用例	实测诊断或行为	退出状态
合法调用	`square(3)`，无诊断	0
未声明名字	`use of undeclared identifier 'unknown'`	1
参数类型错误	`no matching function for call to 'square'`	1
缺少表达式	`int x = ;, expected expression`	1

4. 中间代码与控制流

Clang 自动生成的阶乘 LLVM IR 共 111 行。未优化的局部变量对应 `alloca`，读写对应 `load/store`；有符号循环条件对应 `icmp sle` 和条件 `br`，循环体包含 `mul nsw` 和加一操作。`cin/cout` 变为 C++ 流对象及带符号修饰的函数调用，因此同样的阶乘运算之外还包含运行库接口和静态初始化。

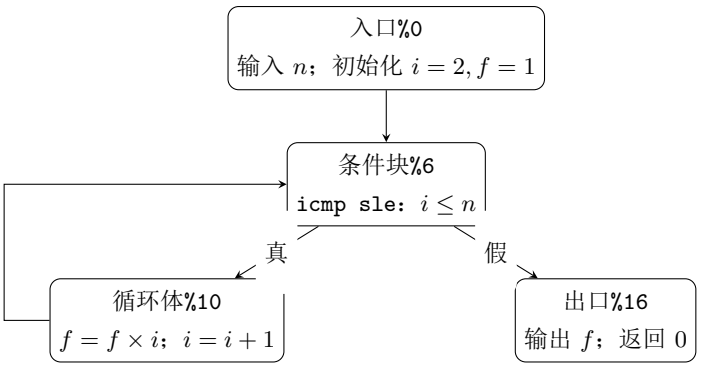


图 4 自动生成 CFG 的四个基本块；省略库调用与变量访存细节

图 4 的入口块初始化 $i=2, f=1$，条件块判断 $i \le n$，真分支进入循环体并返回条件块，假分支执行输出后返回。基本块的边界由控制流决定，不与源语言的花括号一一对应：一个 while 语句就需要条件、循环体和出口等多个基本块。

5. 优化：用具体变换解释 pass 输出

本实验单独运行 mem2reg 和 instcombine，并比较每一步输出的函数体，定位具体优化变换。阶乘的默认 O0 IR 带 `optnone`，直接运行优化可能跳过该函数。因此对 calc(x) 中的 x+0、乘一和加三依次执行两个 pass，并显式关闭该属性。

表 5 受控函数的实际指令数量变化

产物	alloca	load	store	add	mul
原始 IR	3	3	3	2	1
mem2reg 后	0	0	0	2	1
再经 instcombine	0	0	0	1	0

mem2reg 将可提升的栈变量改为 SSA 值，消除了本例的内存访问；instcombine 消除加零与乘一，将函数简化为 $x+3$。对照函数体可以直接看到访存和冗余算术指令的消除。

6. 代码生成：x86 与 RISC-V 的对应关系

分别使用原生 G++ 和课程交叉工具链的 `-S` 获得目标汇编，再对照同一段阶乘循环中的乘法和加一操作。

两段汇编完成相同的变量更新，但操作数的组织方式不同。x86 的 `imull` 可以带内存源操作数；RISC-V 的计算采用寄存器操作数，`lw/sw` 负责访存。RV64 的 `mulw/addiw` 保留 int 的 32 位运算宽度，再符号扩展到 64 位。生成代码还包含栈帧、寄存器保存和 C++ 库函数调用。指令数量不同不能单独推出性能优劣，还取决于优化级别、处理器实现和运行库成本。

```
clang++ -O0 -S fact.cpp -o fact_clang_x86.s
riscv64-unknown-elf-g++ -O0 -S -mcmodel=medany fact.cpp -o fact_rv64.s
grep -n -A 5 'imull' fact_gcc_x86.s
grep -n -B 3 -A 5 'mulw' fact_rv64.s
30:     imull   -16(%rbp), %eax
31-     movl    %eax, -12(%rbp)
32-     addl    $1, -16(%rbp)
33-.L2:
34-     movl    -20(%rbp), %eax
35-     cmpl    %eax, -16(%rbp)
31-     lw      a5,-24(s0)
32-     mv      a4,a5
33-     lw      a5,-20(s0)
34:     mulw    a5,a4,a5
35-     sw      a5,-24(s0)
36-     lw      a5,-20(s0)
37-     addiw   a5,a5,1
38-     sw      a5,-20(s0)
39-.L2:
```

图 5 x86 与 RV64 的循环指令比较

7. 汇编器：机器码、符号表和重定位

先生成汇编，再将.s 输入交给汇编器，单独观察目标文件生成过程。

反汇编显示每条指令对应的机器码字节；符号表中 main 为已定义代码符号，标准流对象及成员函数仍是未定义外部符号。流对象 cin 的引用使用 `R_X86_64_REX_GOTPCRELX`，流提取函数调用使用 `R_X86_64_PLT32`。

汇编器能确定当前目标文件内的指令编码和局部布局，但外部函数的最终地址尚不可知。它记录符号、重定位类型、偏移与附加值，交给链接阶段处理。因此.o 是可重定位文件，不是仅将汇编文本换成二进制的可执行程序。

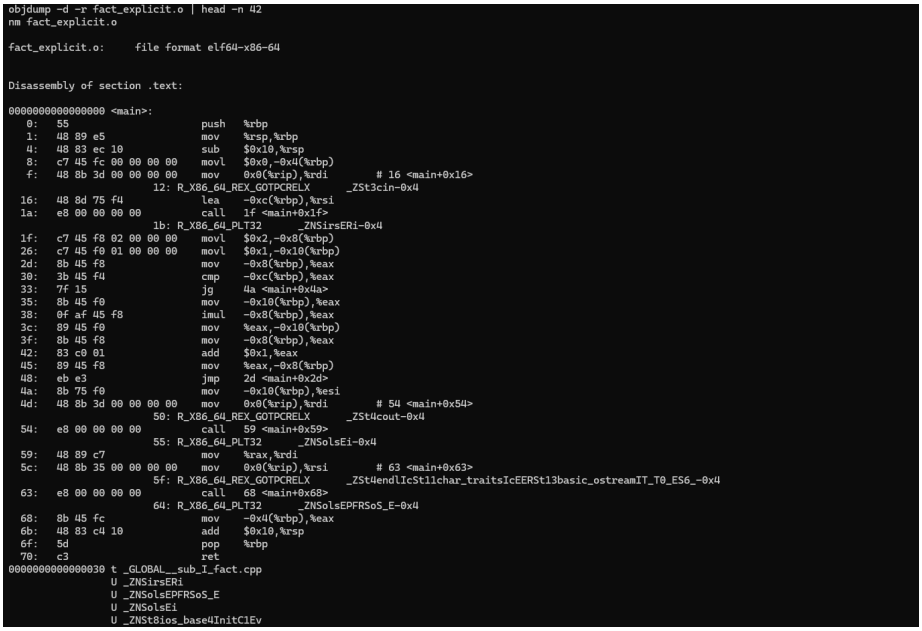


图 6 目标文件机器码、符号表与重定位

8. 链接器：同平台静态与动态链接对照

对同一 x86 程序使用 O0 级别，比较动态、静态和带调试信息的构建。

表 6 同平台构建的实测文件大小

产物	文件字节数	输入 5 的输出
动态链接	16624	120
静态链接	2403984	120
带调试信息	33504	120

动态版的 ldd 显示 libstdc++、libc、libm 和 libgcc_s 等依赖，静态版提示 “not a dynamic executable”。链接器安排输入节的布局、处理重定位并解析符号；静态链接会从归档库提取需要的目标成员，动态链接通常保留 PLT / GOT 和动态符号信息，由动态加载器参与运行时解析。输入 5 时，三种构建均输出 120。

9. 进一步探索：调试信息与优化级别

加入 `-g` 后，`readelf -S` 可见.debug_info、.debug_line 等节；`objdump -d -S` 能在反汇编中插入源码。带调试信息的文件变大，但该变化主要来自调试元数据，不代表执行指令同比增多。

分别使用 O0、O1、O2 和 Os 编译，统计文本行数，结果如下：

文本规模同时受到注释、属性和代码变换影响，因此结合具体指令变化分析优化行为。

10. 自动并行化与运行验证

`par.c` 对 40000 个元素初始化并求和，串行版与自动并行版均输出 799980000；并行版汇编包含一次 `GOMP_parallel` 调用，支持编译器引入 GNU OpenMP 并行执行的判断。并行构建使用 `-ftree-parallelize-loops=4 -pthread`。

表 7 阶乘程序不同优化级别的文本规模

fact 产物	O0	O1	O2	Os
LLVM IR 行数	111	163	250	152
RISC-V 汇编行数	66	49	88	45

```
ddn@LAPTOP-DLPJJ651:~/biany/prep/report_runs$ gcc -O2 par.c -o par_serial
gcc -O2 -ftree-parallelize-loops=4 par.c -o par_omp -pthread
gcc -O2 -ftree-parallelize-loops=4 -S par.c -o par_omp.s -pthread
./par_serial
./par_omp
grep 'GOMP_parallel' par_omp.s
799980000
799980000
        call    GOMP_parallel@PLT
```

图 7 串行与自动并行化的运行结果

阶乘与数组初始化不同，循环中的 f 依赖前一次迭代结果，即 $f_i = f_{i-1} \times i$，不能简单把每次迭代分给独立线程执行。其输入输出位于循环外，跨迭代数据依赖才是分析循环变换时需要考虑的因素。

将输入 5 送入不同构建方式得到的可执行程序，均输出 120，说明本组构建保持了阶乘计算结果。运行验证与前面的代码结构分析相互补充。

（二） 熟悉 LLVM IR 中间语言

1. 实验方法

设计六组 SysY 程序，分别编写 LLVM IR 与 RISC-V 汇编。整数和浮点数分别用 i32、float 表示；局部寄存器遵循 SSA，控制流合流使用 phi，内存变量通过 load/store 访问。

2. 类型、值与源语言语义的对应

手写 IR 将源语言中的变量、条件和地址分别表示为有类型的值。比较指令产生 i1，而输出函数接收 i32，因此布尔结果在输出或参与整数运算前使用 zext。块作用域中的同名变量分配不同的值或存储位置，离开内层块后继续引用外层变量。

表 8 六组示例中的类型与操作映射

源语言对象	本实验的 IR 表示	对应操作
int 及整数常量	i32	add、sub、mul、sdiv、srem
条件与逻辑结果	i1；需要时扩展为 i32	icmp、fcmp、zext、br
float 及浮点参数	float	fadd、fmul、sitofp、fptosi
数组对象与形参	数组类型、ptr 地址	alloca、getelementptr、load
可变全局 hits	全局 i32 存储	load / store 保留调用副作用

例如输入 $n = -7$，sdiv 的 $n/2$ 为 -3，srem 的 $n\%3$ 为 -1，算术式输出 1；随后输出 7、-7、-7，检查单目运算与外层变量值。算术与地址操作分别使用适当的位宽，使变量计算和数组寻址具有明确类型。

3. 编译与验证流程

使用 Clang/LLVM 检查、优化并编译 IR，链接 getint、putint、putch 完成输入输出。验证脚本比较参考程序、手写 IR 和手写汇编的标准输出及退出码。

```
python3 prep/llvm_ir/verify.py
llvm-as sample.ll -o sample.bc
opt -passes=verify -disable-output sample.ll
opt -S '-passes=default<O2>' sample.ll -o sample_O2.ll
llc -mtriple=riscv64-linux-gnu -mattr=+m,+a,+f,+d,+c \
    -target-abi=lp64d sample.ll -o sample_generated_rv.s
```

4. 示例设计与特性覆盖

表 9 本次手写 LLVM IR 的实验设计

示例	主要覆盖	输入数
arithmetic	加减乘除与余数、单目正负、常量、块遮蔽、逻辑非转整数	6
short_circuit	if/else、逻辑与或短路、全局副作用、自定义函数	6
loop_control	while、continue、break、合流与多条回边	6
functions	带参函数、递归、整数返回、void 函数及返回	5
array_float	二维数组、部分初始化、数组形参、浮点运算与转换	5
promote_demo	内存形式循环及 mem2reg 对照	5

算术例子输入 n，计算 $(n+3)\times 2-n/2+n\%3+7$，并分别输出 $-n$、$+n$、n+!n。内层同名变量固定为 7，外层 n 在后续输出中仍保持原值，用于验证遮蔽而非覆盖。负数整除按向零截断计算，余数为 $n-\mathrm{trunc}(n/3)\times 3$。逻辑非先由 icmp 产生 i1，再由 zext 转为 i32 后参与加法。

5. 短路控制流与可观测副作用

短路例子包含 `n != 0 && 12 / n > 2` 和 `n == 0 || touch()`。touch 每次调用将全局 hits 加一。以 n=0 为例，正确程序既不除以零，也不调用 touch，最终 hits=0；非零输入则调用 touch 一次。

```
alghy@LAPTOP-FPD8G88P:/mnt/d/vscodefile/编译系统上机大作业$ scripts/report-evidence.sh short
+ opt -passes=verify -disable-output prep/llvm_ir/short_circuit.ll
+ sed -n 14,35p prep/llvm_ir/short_circuit.ll
entry:
  %n = call i32 @getint()
  %nonzero = icmp ne i32 %n, 0
  br i1 %nonzero, label %and.rhs, label %and.false
and.rhs:
  %q = sdiv i32 12, %n
  %large = icmp sgt i32 %q, 2
  br i1 %large, label %and.true, label %and.false
and.true:
  br label %and.join
and.false:
  br label %and.join
and.join:
  %and.value = phi i32 [1, %and.true], [0, %and.false]
  call void @putint(i32 %and.value)
  call void @putch(i32 10)
  %zero = icmp eq i32 %n, 0
  br i1 %zero, label %or.true, label %or.rhs
or.rhs:
  %called = call i32 @touch()
  %truth = icmp ne i32 %called, 0
  br i1 %truth, label %or.true, label %or.false
+ clang prep/llvm_ir/short_circuit.ll prep/manual/sylib.c -Wno-override-module -o build/report-evidence/short
+ printf '0\n'
+ build/report-evidence/short
0
2
0
+ printf '4\n'
+ build/report-evidence/short
1
2
1
+ set +x

Completed successfully (exit status 0).
```

图 8　短路 IR 验证与输入 0、4 的实际终端运行结果

图 8 中，右操作数置于独立基本块中，由分支决定是否执行。终端中输入 0 和 4 分别输出 0、2、0 与 1、2、1，验证了受保护除法及 touch 的调用次数。测试 n=-3、0、1、4、5、12，覆盖受保护分母、条件真/假和副作用次数。

6. 循环合流与 SSA 值传递

循环例子逐个处理 1 到 n，跳过 2，在数值超过 5 时退出。continue 的边直接回到循环头，break 的边直接进入出口。循环头分别对 i 和 sum 建立 phi，continue 边保持 sum，普通边传入累加后的值。出口的 phi 对正常条件退出和 break 退出分别提供值。

```
header:
  %i = phi i32 [0, %entry], [%next, %skip], [%next, %accumulate]
  %sum = phi i32 [0, %entry], [%sum, %skip], [%added, %accumulate]
  %more = icmp slt i32 %i, %n
  br i1 %more, label %body, label %done
done:
  %result = phi i32 [%sum, %header], [%sum, %break.test]
```

LLVM verifier 检查类型、前驱和支配关系，运行测试检查输出。输入 n=2 时只累加 1，n=5 或 n=8 时结果均为 13，分别触发 continue 与 break。函数例子进一步使用递归阶乘和 void 输出函数，检查参数、call 和 ret 的对应关系。

phi 的每个输入由“值、前驱块”配对，进入当前块时按实际前驱选择。循环首次进入时，i

和 sum 都来自 entry；跳过 2 时，回边更新 i 并保留原 sum；执行累加时，两者分别取 next 和 added。

表 10 循环头 phi 与控制流边的对应关系

进入 header 的前驱	i 的输入	sum 的输入
entry	0	0
skip（continue）	next	原 sum
accumulate（普通回边）	next	added

输入 8 时，在处理数值 6 的 break 路径离开循环，出口选择当前 sum=13，不会执行对 6 的累加。这里同时区分了循环条件退出与循环体内的提前退出。

7. 递归函数与 void 返回

factorial 的参数、返回值和乘法均为 i32。基例直接返回 1，递归分支计算 $n-1$，调用自身后将返回值与当前层的 n 相乘。以下为实际手写函数：

```
define i32 @factorial(i32 %n) {
entry:
  %base = icmp sle i32 %n, 1
  br i1 %base, label %small, label %recursive
small:
  ret i32 1
recursive:
  %previous = sub i32 %n, 1
  %child = call i32 @factorial(i32 %previous)
  %result = mul i32 %n, %child
  ret i32 %result
}
```

递归调用产生新的 child 值，当前层参数 n 在调用返回后仍用于乘法。IR 中无需手动保存返回地址，目标代码生成阶段根据调用约定完成寄存器和栈帧安排。emit 的函数类型为 void(i32)，内部依次调用 putint 和 putch，再以 ret void 结束；main 调用 emit 后以 ret i32 0 返回进程状态。输入 0、1 检查基例，5、7、12 检查递归计算，输出分别为 1、1、120、5040、479001600。

8. 二维数组寻址与浮点转换

数组/浮点例子将 `int a[2][3]=1,2,3,4` 表示为二维数组对象，初始化第二行剩余元素为零。数组实参传地址，按 `[3 x i32]` 的行类型通过 getelementptr 寻址，两行和为 10。浮点过程为 $\text{twice}(1.5n+0.5)=3n+1$，使用 sitofp、fmul、fadd、fptosi 和 fcmp，既验证 int/float 转换，也验证 float 参数和返回值。

row_sum 将行、列索引扩展为 i64 后计算元素地址。其 getelementptr 的源元素类型为 `[3 x i32]`，第一层索引跨越整行，第二层索引定位行内元素。实际地址片段为：

```
%j64 = sext i32 %j to i64
%element = getelementptr [3 x i32], ptr %a, i64 %row64, i64 %j64
%value = load i32, ptr %element, align 4
```

getelementptr 只计算地址，load 才读取元素；本例的字节偏移为 $12r + 4j$。两行分别得到 6 和 4，合计 10，第二行的两个零验证了部分初始化的补零行为。

浮点路径先将 n 转为 float，计算 $1.5n + 0.5$，再通过带浮点参数和返回值的 twice 加倍。负输入 $n = -3$ 得到 -8.0，fptosi 输出 -8，正值判断输出 0；输入 2 得到 7.0，对应整数 7 与判断值 1。数组总和、转换结果和布尔结果分别输出，便于区分寻址错误与计算错误。

9. mem2reg 观察与结果验证

另手写一份以 alloca/load/store 保存循环变量的求和程序，使用 LLVM 自带 `opt -passes=mem2reg` 得到可提升变量的寄存器形式。转换前为 2 个 alloca、3 个 load、4 个 store、0 个 phi；转换后这些局部访存操作均消失，形成 2 个 phi。对 n=-3、0、1、5、10 分别运行，转换前后输出一致。

优化后的两个 phi 分别对应 index 与 sum。循环开始时 $i = 0, s = 0$，每次迭代执行 $s' = s+i$、$i' = i + 1$，因此循环头保持 $s = i(i - 1)/2$。正输入 n 的结果为 $n(n - 1)/2$，非正输入直接输出 0。实测 n=-3、0、1、5、10 的输出分别为 0、0、0、10、45。这一对照同时检查了局部存储消除、循环回边与计算结果的保持。

优化前后的指令对照见图 9。

```
alghy@LAPTOP-FPD8G88P:/mnt/d/vscodefile/编译系统上机大作业$ scripts/report-evidence.sh mem2reg
+ opt -S -passes=mem2reg prep/llvm_ir/promote_demo.ll -o build/report-evidence/promoted.ll
+ grep -nE '\b(alloca|load|store|phi)\b' prep/llvm_ir/promote_demo.ll build/report-evidence/promoted.ll
prep/llvm_ir/promote_demo.ll:7:  %index = alloca i32, align 4
prep/llvm_ir/promote_demo.ll:8:  %sum = alloca i32, align 4
prep/llvm_ir/promote_demo.ll:10:  store i32 0, ptr %index, align 4
prep/llvm_ir/promote_demo.ll:11:  store i32 0, ptr %sum, align 4
prep/llvm_ir/promote_demo.ll:14:  %i = load i32, ptr %index, align 4
prep/llvm_ir/promote_demo.ll:18:  %old = load i32, ptr %sum, align 4
prep/llvm_ir/promote_demo.ll:20:  store i32 %added, ptr %sum, align 4
prep/llvm_ir/promote_demo.ll:22:  store i32 %next, ptr %index, align 4
prep/llvm_ir/promote_demo.ll:25:  %result = load i32, ptr %sum, align 4
build/report-evidence/promoted.ll:16:  %sum.0 = phi i32 [ 0, %entry ], [ %added, %body ]
build/report-evidence/promoted.ll:17:  %index.0 = phi i32 [ 0, %entry ], [ %next, %body ]
+ clang build/report-evidence/promoted.ll prep/manual/sylib.c -Wno-override-module -o build/report-evidence/promoted
+ printf '10\n'
+ build/report-evidence/promoted
45
+ set +x

Completed successfully (exit status 0).
```

图 9 mem2reg 指令对照与输入 10 输出 45 的终端截图

（三） 熟悉汇编语言

1. 手写 RISC-V 汇编的组织与调用约定

六组示例采用 RV64GC 指令集和 LP64D 调用约定。整数参数和返回值使用 a0 等参数寄存器，浮点函数的单精度参数和返回值使用 fa0。遵循 RISC-V psABI，函数调用处保持 sp 的 16 字节对齐，使用 s 寄存器的函数保存并恢复相应值；非叶函数保存返回地址 ra。跨调用保留的数据存入 s 寄存器或栈中。

SysY 的 int 是 32 位，而 RV64 的整数寄存器是 64 位。因此使用 addw、addiw、subw、mulw、divw、remw 等指令处理整数计算，lw 读取数组元素，并保持 32 位结果的符号扩展。寻址和 sp 调整则使用 64 位指令。伪指令 call、ret、li、la 由汇编器展开。

表 11 手写汇编与同名 SysY、LLVM IR 示例的对应关系

文件	实现重点
arithmetic_rv.s	word 算术、向零截断除法、有符号余数、跨输出调用保存输入
short_circuit_rv.s	受保护除法、分支短路、全局 hits 与 touch 副作用
loop_control_rv.s	continue 回边、break 出口、循环状态寄存器
functions_rv.s	递归栈帧、保存 ra 与原参数、void 输出函数
array_float_rv.s	二维数组行步长、补零、float ABI、显式截断转换
promote_demo_rv.s	寄存器求和循环，与内存形式 LLVM IR 对照

2. 短路、循环与递归的目标代码表达

短路例子先检查 n，再进入右侧表达式。以下为受保护除法的完整控制片段：

```
    li a0,0
    beqz s0,.Land_done
    li t0,12
    divw t0,t0,s0
    li t1,2
    slt a0,t1,t0
.Land_done:
    call putint
```

其中 s0 保存 n。n=0 时直接跳过 divw；另一个逻辑或条件在 n=0 时直接进入真分支，不会调用 touch。全局 hits 通过 la/lw/sw 访问，输出次数与 LLVM IR 版本一致。循环例子用寄存器保存 i 与 sum，continue 跳回条件检查，break 跳到输出出口。LLVM IR 中的 phi 对应不同路径对状态寄存器的更新。

本例用 t0 保存 i、t1 保存 sum，输入上限保留在 a0。循环内不调用其他函数，因此这些寄存器在整个迭代过程中保持可用。遇到 2 时直接回到条件检查，t1 不变；遇到大于 5 的值时跳到出口，只在普通路径执行 addw。这与 IR 中 skip、break.test、accumulate 三条路径一一对应。

表 12 同一寄存器循环中的条件退出、continue 与 break

输入上限	实际累加项	输出	控制路径
0	无	0	初始条件退出
2	1	1	跳过 2 后退出
5	1、3、4、5	13	正常条件退出
8	1、3、4、5	13	处理 6 时 break

递归 factorial 的非基本情形保存原参数和 ra，再调用自身。图 10 展示完整函数、交叉编译命令及 QEMU 输入 7 输出 5040 的运行结果。

```
alghy@LAPTOP-FPD8G88P:/mnt/d/vscodefile/编译系统上机大作业$ scripts/report-evidence.sh rv
+ sed -n 5,21p prep/llvm_ir/handwritten/functions_rv.s
factorial:
    li t0,1
    ble a0,t0,.Lbase
    addi sp,sp,-16
    sd ra,8(sp)
    sd s0,0(sp)
    mv s0,a0
    addiw a0,a0,-1
    call factorial
    mulw a0,s0,a0
    ld s0,0(sp)
    ld ra,8(sp)
    addi sp,sp,16
    ret
.Lbase:
    li a0,1
    ret
+ riscv64-linux-gnu-gcc -static -march=rv64gc -mabi=lp64d prep/llvm_ir/handwritten/functions_rv.s prep/manual/sylib.c -o build/report-evidence/functions_rv
+ printf '7\n'
+ qemu-riscv64 build/report-evidence/functions_rv
5040
+ set +x

Completed successfully (exit status 0).
```

图 10 递归阶乘的手写汇编、交叉编译与 QEMU 运行截图

基本情形 n<=1 直接返回 1，不建立额外栈帧。emit 不返回计算值，但它调用输出库，也必须保存 ra；源程序的 void 返回对应恢复调用现场后 ret。测试 n=0、1、5、7、12，既检查递归出口也检查多层返回。

3. 栈帧布局与跨调用数据保存

factorial 的非基例在 16 字节栈帧中保存 ra 和 s0，s0 持有当前层参数。递归返回时 a0 已被子调用结果覆盖，所以乘法使用恢复前仍在 s0 中的原参数。以 n=3 为例，子调用依次返回 1、2，当前层再计算 $3 \times 2 = 6$；每层恢复 s0 和 ra 后释放自己的栈帧，使控制流返回正确的调用位置。

数组示例还需要在多次函数调用之间保存数组、浮点结果和中间整数。其 main 实际使用如下 64 字节栈帧：

表 13 array_float 的 main 栈帧布局

相对 sp 的偏移	宽度	用途
0、4、8、12、16、20	各 4 字节	二维数组六个 i32 元素
24	4 字节	scaled 的单精度值
28–39	12 字节	未用于保存有效数据的间隔
40、48	各 8 字节	保存 s1、s0
56	8 字节	保存 ra

调用 row_sum 后，第一行结果转存 s0，随后第二次调用产生的 a0 与 s0 相加。浮点结果提前用 fsw 保存到偏移 24，整数截断值放入 s1；输出数组和整数后，再通过 flw 恢复浮点值完成正值判断。这种安排使计算数据在所需调用期间保持有效，并在函数返回前恢复保存寄存器。

4. 数组布局与浮点数实现

二维数组采用行优先布局，每行含三个 i32 元素，因此行步长为 12 字节。row_sum 接收 a0 中的数组首地址和 a1 中的行下标，用 $\text{base} + 12 \times \text{row} + 4 \times \text{column}$ 计算元素地址。main 的 64 字节栈帧中，偏移 0–20 保存六个元素，24 保存 scaled，40、48、56 分别保存 s1、s0、ra；未初始化的两个数组元素显式写零。输入和输出库调用使用独立栈帧，不覆盖调用者的数组区域。

整数输入通过 fcvt.s.w 转换为 float，fmul.s 与 fadd.s 计算 $1.5n + 0.5$，twice 在 fa0 中收参并返回 $3n + 1$。fcvt.w.s 的 rtz 模式显式实现 float 到 int 的向零截断。scaled 在后续 row_sum、putint、putch 调用期间保存在栈上，最后重新加载并用 flt.s 比较。

5. 手写与 LLVM 后端输出的对照

验证过程同时保存 llc 生成的 O0、O2 汇编。在 loop_control 的 O0 文件中，循环状态使用 a0、a4、a5 等寄存器，并形成 header.outer 与 body 对应的循环标签；手写版本直接用 t0/t1 更新 i 与 sum，保留与源程序对应的标签。O2 IR 则出现 body.peel、next.lcssa 等块和值，并在处理跳过 2 的路径上形成常量 3。生成的目标代码因此不再逐块复刻原始手写 IR。

这一对照分为三个层次：先检查源语言的跳过与退出规则，再检查 IR 的合流值，最后检查目标代码中的分支、寄存器更新和调用现场。六组循环输入在原始 IR、O2 IR 和手写汇编路径上输出一致，说明这些不同的代码组织保持了测试输入上的计算结果。

6. 汇编、链接和同输入等价性验证

验证脚本对每个手写文件执行静态链接并由 QEMU 运行，例如：

```
riscv64-linux-gnu-gcc -static -march=rv64gc -mabi=lp64d \
  prep/llvm_ir/handwritten/functions_rv.s prep/manual/sylib.c \
  -o build/prep-llvm-ir/functions_handwritten_rv
printf '7\n' | qemu-riscv64 build/prep-llvm-ir/functions_handwritten_rv
# stdout: 5040, process exit status: 0
```

汇编器生成目标代码，链接器解析库函数，QEMU 执行 RV64 程序。

表 14 实际执行的代表结果，逗号表示换行分隔的输出

示例	代表输入	源程序/IR/手写汇编 stdout	验证重点
arithmetic	-7	1, 7, -7, -7	负数除法与余数
short_circuit	0 / 4	0, 2, 0 / 1, 2, 1	短路与副作用
loop_control	2 / 8	1 / 13	continue / break
functions	7	5040	递归与返回
array_float	-3 / 2	10, -8, 0 / 10, 7, 1	数组、浮点与转换
promote_demo	5	10	循环累加

每个输入比较八条路径：C 参考程序本机/RV64、手写 IR 本机/RV64 （原始与 O2）、lli、手写 RV64 汇编。33 组输入共 264 项，加 mem2reg 的 5 项为 **269/269 通过**。arithmetic、short_circuit、loop_control 各 48 项，functions 与 array_float 各 40 项，promote_demo 含 mem2reg 共 45 项。全部输出逐字比较，并检查进程退出码为 0。

三、 进阶要求

（一） 观察 AI 编译器 MLIR 中端处理过程

1. 研究方法

依据 AscendNPU IR 官方 VecAdd 示例和方言文档分析中端层级，并使用 MLIR 21.1.8 构造上游 VecAdd，执行逐层降低和 CPU 运行验证。

2. AscendNPU IR VecAdd 的方言与层级

官方快速入门的 VecAdd 使用 i16 元素，并区分 GM 与 UB 地址空间。输入先搬入片上缓冲，再进行向量加法，最后搬回输出；设备入口用 HACC 属性表达。编译后还需要 CANN runtime 注册和启动设备程序。

架构说明把 HFusion、HIVM、HACC 分为高层融合、硬件相关计算/搬运/同步、异构执行表达等职责。结合 HIVM 文档，可以看到片上内存和计算流水信息在特定层被显式保留。官方快速入门从 HIVM 级程序开始，体现片上缓冲、向量计算和数据搬运的配合。

表 15 两种路线的职责与证据差异

比较点	上游 CPU 实验	Ascend 官方文档路线
高层计算	Linalg 通用逐元素计算	HFusion 等较高层表达
硬件相关操作	普通 MemRef 访存	HIVM 中的计算、搬运、同步
内存空间	本机普通内存	GM、UB 等硬件空间
执行接口	MLIR 生成 C 包装函数	HACC 入口及 CANN runtime
证据	本机各层输出与运行检查	官方示例和架构文档分析

3. 可执行的上游 VecAdd 设计

独立设计的 `prep/mlir/vecadd.mlir` 使用三个 `memref<16xi16>` 参数。Linalg 的三个恒等索引映射表示 $c[i] = a[i] + b[i]$，迭代属性 parallel 表明各元素计算互相独立。本实验将其降低为标量循环。

```
linalg.generic {
  indexing_maps = [affine_map<(i)->(i)>,
                   affine_map<(i)->(i)>, affine_map<(i)->(i)>],
  iterator_types = ["parallel"]
} ins(%a, %b : memref<16xi16>, memref<16xi16>)
  outs(%c : memref<16xi16>) {
^bb0(%x: i16, %y: i16, %old: i16):
  %sum = arith.addi %x, %y : i16
  linalg.yield %sum : i16
}
```

输入数组设为 $a[i] = i$、$b[i] = 1$（$0 \le i < 16$），C 驱动逐项检查输出。

4. 逐层 lowering 与检查点

上游 MLIR 文档介绍结构化 SCF 与非结构化 CF 的区分。本次根据工具可用 pass 执行如下管线，每一步均保存文件：

```
mlir-opt vecadd.mlir -o 00_linalg.mlir
mlir-opt 00_linalg.mlir --convert-linalg-to-loops -o 01_scf.mlir
mlir-opt 01_scf.mlir --convert-scf-to-cf -o 02_cf.mlir
mlir-opt 02_cf.mlir --convert-to-llvm \
    --reconcile-unrealized-casts -o 03_llvm.mlir
mlir-translate --mlir-to-llvmir 03_llvm.mlir -o 04_llvm.ll
```

上述每一步的输出都由验证脚本保存为独立文件，供后续逐层对照。

表 16 MLIR 降低阶段与操作方言

检查点	行数	实际观察到的操作方言
00_linalg	12	Linalg、Arith、Func
01_scf	15	SCF、MemRef、Arith、Func
02_cf	21	CF、MemRef、Arith、Func
03_llvm	67	LLVM

首先 generic 被展开为 `scf.for`，循环体出现两个 memref.load、一次 arith.addi 和一次 memref.store。接着循环被拆成比较、条件分支、循环体和回边，迭代变量通过块参数传递；结构化循环这个概念不再显式保留。两层输出的对照见图 11。

```
alghy@LAPTOP-FPD8G88P:/mnt/d/vscodefile/编译系统上机大作业$ scripts/report-evidence.sh mlir
+ cat prep/mlir/results/01_scf.mlir
module {
  func.func @vecadd(%arg0: memref<16xi16>, %arg1: memref<16xi16>, %arg2: memref<16xi16>) attributes {llvm.emit_c_interface} {
    %c0 = arith.constant 0 : index
    %c16 = arith.constant 16 : index
    %c1 = arith.constant 1 : index
    scf.for %arg3 = %c0 to %c16 step %c1 {
      %0 = memref.load %arg0[%arg3] : memref<16xi16>
      %1 = memref.load %arg1[%arg3] : memref<16xi16>
      %2 = arith.addi %0, %1 : i16
      memref.store %2, %arg2[%arg3] : memref<16xi16>
    }
    return
  }
}

+ cat prep/mlir/results/02_cf.mlir
module {
  func.func @vecadd(%arg0: memref<16xi16>, %arg1: memref<16xi16>, %arg2: memref<16xi16>) attributes {llvm.emit_c_interface} {
    %c0 = arith.constant 0 : index
    %c16 = arith.constant 16 : index
    %c1 = arith.constant 1 : index
    cf.br ^bb1(%c0 : index)
  ^bb1(%0: index):  // 2 preds: ^bb0, ^bb2
    %1 = arith.cmpi slt, %0, %c16 : index
    cf.cond_br %1, ^bb2, ^bb3
  ^bb2:  // pred: ^bb1
    %2 = memref.load %arg0[%0] : memref<16xi16>
    %3 = memref.load %arg1[%0] : memref<16xi16>
    %4 = arith.addi %2, %3 : i16
    memref.store %4, %arg2[%0] : memref<16xi16>
    %5 = arith.addi %0, %c1 : index
    cf.br ^bb1(%5 : index)
  ^bb3:  // pred: ^bb1
    return
  }
}

+ set +x

Completed successfully (exit status 0).
```

图 11 终端中 SCF 与 CF 实际输出文件的对照

再向 LLVM 方言转换时，MemRef 描述符中的已分配地址、对齐地址、偏移、长度和步长被显式展开，生成与 C 调用者衔接的包装函数。最后 `mlir-translate` 才输出真正的 LLVM IR。因此 `llvm.*` 操作所在的 MLIR 文件，与 LLVM 自身的.ll 文件并不是同一种文本格式。

5. 逐层操作的对应与语义保持

在三个恒等索引映射下，同一个迭代变量 i 同时定位 a、b、c 的第 i 个元素。降低为 SCF 后，循环范围为 $[0, 16)$、步长为 1；转为 CF 后，循环头的块参数承接初值 0 和回边上的 $i + 1$，比较结果决定进入计算块或返回块。最终 LLVM IR 的循环头使用 phi i64 表达这一合流，而加法数据仍为 i16。

表 17 本次 VecAdd 各层的操作对应关系

计算要素	中间层表示	LLVM 层表示
逐元素索引	Linalg 恒等索引映射	以迭代变量计算元素地址
循环迭代	scf.for；CF 块参数	phi i64、icmp、br
读取输入	memref.load	getelementptr i16、load i16
加法与写回	arith.addi、memref.store	add i16、store i16
函数出口	func.return	ret void

循环边界、迭代步长和元素运算在各层保持一致；变化主要在于控制流与地址的表达方式。最终文件中三个 getelementptr 的元素类型均为 i16，连续索引相差 2 字节。每次迭代读取两个输入、相加并写入一个输出位置，对应原始 generic 的一个迭代实例。

6. MemRef 描述符与 C 调用接口

高层的 `memref<16xi16>` 同时携带元素类型、维数和长度信息。LLVM 层的一维描述符展开为两个指针、一个偏移、一个长度和一个步长。C 驱动使用与实际生成接口匹配的结构：

```
    int16_t *allocated, *aligned;
    int64_t offset, sizes[1], strides[1];
} MemRef16;
extern void _mlir_ciface_vecadd(MemRef16 *, MemRef16 *, MemRef16 *);
int main(void) {
```

表 18 CPU VecAdd 的一维 MemRef 描述符

描述符字段	本实验设置	含义
allocated、aligned	均为对应数组首地址	分配地址与对齐后的数据地址
offset	0	第一个逻辑元素的偏移
sizes[0]	16	一维数组长度
strides[0]	1	相邻逻辑元素的元素步长

生成的 `_mlir_ciface_vecadd` 接收三个描述符地址，读取各字段后调用内部 vecadd。内部函数将每个描述符展开为五个参数，共 15 个参数。本例为静态长度、零偏移、单位步长，降低后的实际循环直接从 aligned 地址按 i16 元素索引访存。因此，源层的数组接口与底层指针接口之间由生成的包装函数衔接。

在 `03_llvm.mlir` 中，构造与读取描述符使用 llvm.insertvalue、llvm.extractvalue，循环和访存使用 llvm.br、llvm.getelementptr、llvm.load/store。翻译后的 `04_llvm.ll` 则采用 LLVM 原生指令语法，循环索引合流已表现为 phi。从 12 行到 67 行的展开既包含计算循环，也包含描述符组装和 C 接口包装。

7. 末端验证与分析

C 驱动构造与一维 MemRef 对应的描述符，通过 `_mlir_ciface_vecadd` 调用降低后的程序。本次 LLVM verifier 检查通过，Clang 编译链接后输出 1 至 16。图 12 记录 LLVM/RISC-V 的 269 项检查和 VecAdd 的 16 项检查均通过。

```
alghy@LAPTOP-FPD8G88P:/mnt/d/vscodefile/编译系统上机大作业$ scripts/report-evidence.sh results
+ python3 prep/llvm_ir/verify.py
PASS arithmetic: 6 inputs x 8 execution paths
PASS short_circuit: 6 inputs x 8 execution paths
PASS loop_control: 6 inputs x 8 execution paths
PASS functions: 5 inputs x 8 execution paths
PASS array_float: 5 inputs x 8 execution paths
PASS promote_demo: 5 inputs x 8 execution paths
ALL PASS: 269/269; mem2reg: {'before': {'alloca': 2, 'load': 3, 'store': 4, 'phi': 0}, 'after': {'alloca': 0, 'load': 0, 'store': 0, 'p
hi': 2}}
+ python3 prep/mlir/run_lowering.py
+ tail -n 1 build/report-evidence/mlir-run.log
PASS: VecAdd 16/16; 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
+ build/prep-mlir/vecadd
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
+ set +x

Completed successfully (exit status 0).
```

图 12 LLVM/RISC-V 269/269 与 MLIR VecAdd 16/16 的实际验证截图

驱动逐个断言全部 16 个元素正确，退出码 0。

验证驱动将输出数组初始化为零，再设置 $a[i] = i$、$b[i] = 1$。对每个位置比较 $c[i]$ 与 $i + 1$，发现不匹配即输出错误位置并返回 1。本次所有元素通过检查后完整输出 1 至 16，并返回 0，将“各层文件生成成功”与“末端计算正确”连接起来。版本、命令、stderr 和结果保存于 `prep/mlir/results/`。行数从 12 增至 67，主要来自内存描述符与调用接口的展开。

多级 IR 在高层保留索引映射和计算结构，在中层明确循环与访存，最终转换为指针操作和调用接口。各阶段的类型、控制流和内存表示随降低同步转换。

四、 总结与体会

编译阶段输出展示了源程序向 IR、汇编、目标文件和可执行程序的转换。手写 LLVM IR 与 RISC-V 汇编进一步说明短路求值、循环合流、数组布局和调用约定的实现方式。六组程序在八条执行路径上输出一致，连同 mem2reg 对照，269 项检查全部通过。MLIR VecAdd 实验展示了从高层计算结构到循环、分支和指针操作的逐层降低，16 个输出元素全部正确。实验将语言语义、中间表示与目标执行结果建立了对应关系。

动手之后最直接的体会是，容易写错的往往不是算法，而是读代码时会跳过的约定：int 是 32 位而 RV64 的寄存器是 64 位，sp 要保持 16 字节对齐，非叶函数要保存 ra。这类差错通常不会让程序立刻失败，而是给出一个看似合理的结果，所以验证不能只看返回值——短路例子以 touch 的调用次数和全局 hits 计数作为判据。工具也会掩盖问题：O0 生成的函数带 optnone 属性，直接运行 opt 不报错，只是什么都不做，pass 的输出只有与预期的具体变换逐条对照才能确认。MLIR 的逐层降低则表明，高层保留的索引映射与并行语义，在低层被展开成指针、循环和调用接口，丢掉的信息正是上一层优化所需的，这也解释了为什么要比较 llc 输出与手写汇编。此外，两部分分别在 Ubuntu 22.04（LLVM 14、QEMU 7.0.0）与 Ubuntu 26.04（LLVM 21、QEMU 10.2.1）上完成，工具链版本并不一致，同一份手写 IR 在两端并不通用，后续实验宜先固定版本再动手。